



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

INGENIERIA DE SOFTWARE

COMPUTACIÓN GRAFICA

ESTUDIANTES:

TONATO CAJAMARCA MATIAS
ALEXANDER

GUERRERO AUQUILLA JOHN

RUBIO BENALCAZAR NESTOR ARIEL

TEMA:

ELIMINACIÓN DE LÍNEA OCULTA
(HIDDEN-LINE)

DOCENTE:

ING. NANCY JACHO

FECHA:

18 DE ENERO DE 2026

INDICE

I.	Resumen	3
II.	INTRODUCCIÓN.....	3
III.	MARCO TEÓRICO	4
3.1.	Definición de Eliminación de Líneas Ocultas	4
3.2.	Clasificación de los Algoritmos	5
3.2.1.	Algoritmos de Espacio Objeto (Object-Space).....	5
3.2.2.	Algoritmos de Espacio Imagen (Image-Space)	6
3.3.	Coherencia	6
IV.	PRINCIPALES MÉTODOS DE ELIMINACIÓN	7
4.1.	Detección de Caras Traseras (Back-Face Detection/Culling).....	7
4.2.	Algoritmo del Búfer de Profundidad (Depth-Buffer o Z-Buffer).....	9
4.3.	Algoritmo de Warnock (Subdivisión de Área)	10
4.4.	Algoritmo de Línea de Barrido (Scan-Line)	12
V.	MODELO MATEMÁTICO Y ALGORÍTMICO.....	14
5.1.	Modelo Geométrico del Problema	14
5.2.	Sistema de referencia y vector de visión	14
5.2.1.	Vector de visión (cámara).....	14
5.3.	Vectores normales de las superficies	15
5.4.	Producto punto y Criterio de visibilidad	16
5.4.1.	Interpretación Geométrica	16
5.5.	Back-Face Culling (Base matemática del HLR)	16
5.5.1.	Principio Matemático	16
5.6.	Determinación de aristas visibles	17
5.7.	Proyección al plano de la imagen	17
5.7.1.	Proyección Perspectiva	17
VI.	CONCLUSIONES	17
VII.	RECOMENDACIONES	18
VIII.	REFERENCIAS BIBLIOGRÁFICAS.....	19
IX.	ANEXOS.....	19

I. Resumen

La representación de objetos tridimensionales en pantallas bidimensionales presenta desafíos inherentes relacionados con la percepción de profundidad y volumen. Uno de los problemas fundamentales en la graficación por computadora es la ambigüedad visual generada por las representaciones de "malla de alambre" (wireframe), donde todas las aristas de un objeto son visibles simultáneamente, independientemente de su posición relativa al observador. La presente investigación aborda la Eliminación de Líneas Ocultas (HLR, por sus siglas en inglés) como un conjunto de técnicas y algoritmos diseñados para determinar qué líneas o segmentos de líneas son visibles desde un punto de vista específico y cuáles están ocultos por superficies opacas. Se exploran las diferencias entre algoritmos de espacio objeto y espacio imagen, así como la importancia de la coherencia en la eficiencia computacional. Este estudio sienta las bases teóricas necesarias para la posterior implementación de modelos matemáticos y simulaciones en OpenGL.

Palabras clave: Computación gráfica, Eliminación de líneas ocultas, Espacio imagen, Espacio objeto, Proyección 3D.

II. INTRODUCCIÓN

La evolución de la computación gráfica ha estado marcada por la búsqueda constante de realismo y claridad en la representación de entornos virtuales. En las etapas iniciales del diseño asistido por computadora (CAD) y la visualización de datos, los modelos se representaban casi exclusivamente mediante estructuras de alambre. Si bien este método es computacionalmente económico, presenta una desventaja crítica: la falta de señales de oclusión. Sin la eliminación de las partes traseras del modelo, el cerebro humano puede interpretar la orientación del objeto de

manera errónea, invirtiendo la percepción de profundidad (el efecto conocido como "cubo de Necker").

La eliminación de líneas ocultas no es meramente una cuestión estética, sino funcional. En ingeniería y arquitectura, es vital saber qué componentes están delante de otros. El proceso de identificar y eliminar estas líneas (o superficies, en el caso de renderizado sólido) es uno de los problemas clásicos y más intensivos en cuanto a cálculo en la informática gráfica.

Esta investigación tiene como objetivo desglosar los fundamentos teóricos que permiten a una computadora discernir la visibilidad. Se analizarán las metodologías que clasifican las primitivas geométricas según su posición en el eje Z (profundidad) respecto a una cámara virtual. A través de este análisis, se comprenderá cómo los motores gráficos modernos, como OpenGL, gestionan la visibilidad de manera transparente para el usuario final, aunque basándose en los principios algorítmicos que se detallarán a continuación.

III. MARCO TEÓRICO

3.1. Definición de Eliminación de Líneas Ocultas

La eliminación de líneas ocultas se define como el proceso computacional mediante el cual se identifican y suprimen las aristas o segmentos de aristas de un objeto tridimensional que no son visibles desde la posición de una cámara o un observador, debido a que están obstruidos por las superficies opacas del propio objeto o por otros objetos en la escena [1].

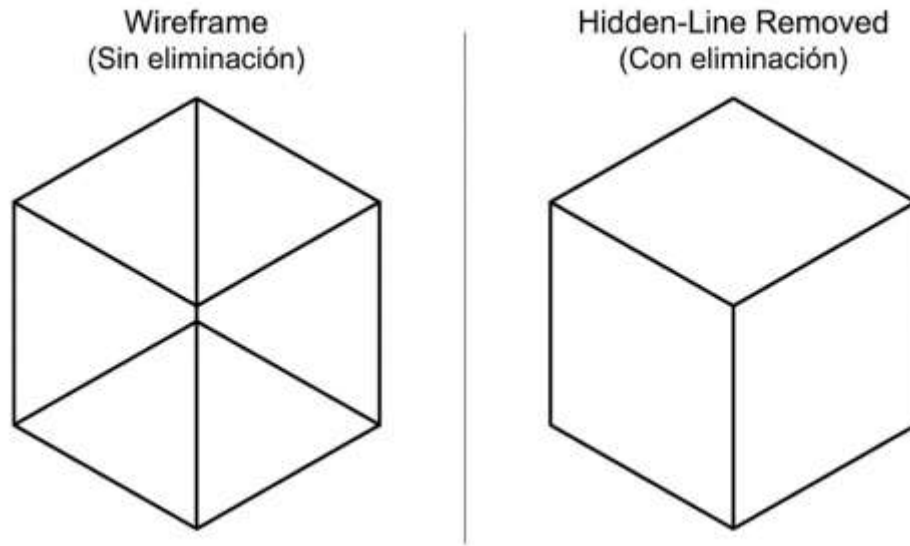


Figura 1. Comparación de visualización

Es crucial distinguir entre la Eliminación de Líneas Ocultas (HLR) y la Eliminación de Superficies Ocultas (HSR). Mientras que la HLR se centra en el dibujo de contornos y mallas (típico en planos técnicos), la HSR se ocupa de la visibilidad de los píxeles en superficies rellenas de color o textura. Sin embargo, muchos autores, como Foley, señalan que los algoritmos subyacentes suelen ser idénticos o variaciones del mismo principio: determinar la visibilidad basada en la profundidad [2].

3.2. Clasificación de los Algoritmos

Para abordar el problema de la visibilidad, la literatura técnica clasifica los algoritmos en dos grandes categorías, dependiendo del dominio en el que operan:

3.2.1. Algoritmos de Espacio Objeto (Object-Space)

Estos algoritmos operan directamente sobre las definiciones geométricas de los objetos (coordenadas físicas en el mundo 3D). Comparan cada objeto o arista con todos los demás objetos de la escena para determinar si existe una obstrucción.

El enfoque general se puede describir como:

"Para cada objeto en la escena, compáralo con todos los demás objetos para determinar cuál es visible."

Según Hearn y Baker, estos métodos son ideales cuando la cantidad de objetos es baja, ya que ofrecen una precisión geométrica muy alta, permitiendo escalar la imagen sin perder calidad (como en los plotters vectoriales) [1]. Sin embargo, su complejidad computacional tiende a crecer cuadráticamente $O(n^2)$, lo que los hace ineficientes para escenas complejas con miles de polígonos.

3.2.2. Algoritmos de Espacio Imagen (Image-Space)

Estos algoritmos determinan la visibilidad punto por punto (o píxel por píxel) en la ventana de proyección. La pregunta que resuelven es:

"Para cada píxel en la pantalla, ¿qué objeto es el más cercano al observador?"

La mayoría de los sistemas gráficos modernos, incluido OpenGL, utilizan variaciones de algoritmos de espacio imagen (como el Z-Buffer) debido a su eficiencia. La complejidad depende de la resolución de la pantalla y no tanto de la complejidad geométrica de los objetos superpuestos. Foley destaca que, aunque pueden sufrir de problemas de "aliasing" (bordes dentados), son mucho más rápidos para renderizado en tiempo real [2].

3.3.Coherencia

Un concepto fundamental citado por Foley para la optimización de estos algoritmos es la "coherencia". La coherencia se basa en la observación de que las propiedades de una escena tienden a ser constantes localmente [2]. Los tipos de coherencia aprovechados incluyen:

- Coherencia de cara: Si una parte de una cara es visible, es probable que el resto de la cara también lo sea.
- Coherencia de profundidad: Las superficies adyacentes suelen tener profundidades similares.
- Coherencia de escaneo: En una línea de barrido (scan-line), los segmentos visibles suelen cambiar poco de una línea a la siguiente.

IV. PRINCIPALES MÉTODOS DE ELIMINACIÓN

Existen diversos métodos para resolver el problema de las líneas ocultas. A continuación, se describen los más representativos desde una perspectiva lógica y conceptual.

4.1. Detección de Caras Traseras (Back-Face Detection/Culling)

Este método, conocido técnicamente como *Back-Face Culling*, representa la primera línea de defensa en la optimización del renderizado 3D. Es el algoritmo más eficiente para objetos convexos individuales, ya que permite descartar geometría antes de que entre en etapas más costosas del pipeline gráfico (como la rasterización). Su funcionamiento se basa estrictamente en la orientación geométrica de las superficies respecto a la cámara. La premisa lógica es simple: en un objeto sólido y opaco, cualquier cara cuya normal (el vector perpendicular a la superficie) apunte en dirección opuesta al observador está siendo obstruida por las caras frontales del propio objeto. Por lo tanto, calcular su color o iluminación es un desperdicio de recursos computacionales.

Para determinar esta orientación sin ambigüedades, los sistemas gráficos modernos como OpenGL utilizan el concepto de "Orden de los Vértices" (Winding Order). Cuando se define un triángulo, el orden en que se declaran sus vértices (horario o antihorario) determina hacia dónde apunta su normal implícita. Si al proyectar el triángulo en la pantalla (espacio imagen), el orden de sus vértices sigue una dirección antihoraria, se considera que la cara está "mirando" al frente; si es horaria, es una cara trasera. Matemáticamente, esto se confirma mediante el producto punto entre el vector normal de la superficie y el vector de visión. Si el resultado es negativo (dependiendo del sistema de coordenadas), la superficie se considera oculta y el motor gráfico la descarta inmediatamente.

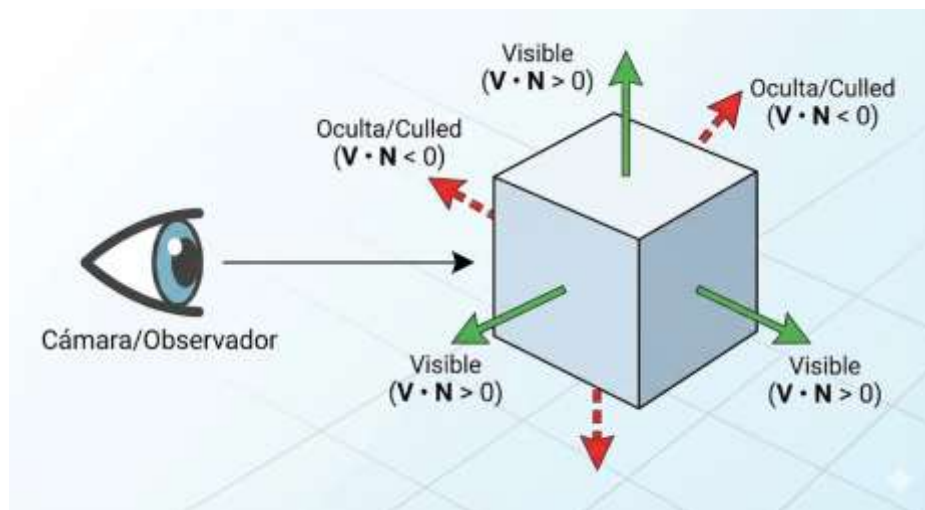


Figura 2. Prueba de visibilidad mediante vectores normales

Hearn y Baker explican que este método elimina aproximadamente el 50% de las superficies de una escena cerrada en promedio, duplicando efectivamente la velocidad de renderizado. Sin embargo, su limitación principal radica en que opera a nivel de polígono individual. No resuelve el problema de la oclusión global; es decir, no puede determinar si un objeto cercano está tapando a un objeto lejano (ambos orientados hacia el frente), ni puede manejar objetos cóncavos donde

una cara frontal se pliega sobre sí misma y oculta a otra cara frontal. Por esta razón, el *Back-Face Culling* rara vez se usa solo; generalmente se combina con algoritmos de espacio imagen como el Z-Buffer [1].

4.2.Algoritmo del Búfer de Profundidad (Depth-Buffer o Z-Buffer)

Aunque es técnicamente un método de espacio imagen diseñado originalmente para la eliminación de superficies ocultas, el algoritmo del Z-Buffer se ha convertido en el estándar indiscutible de la industria para la visibilidad general, siendo implementado directamente en el hardware de las tarjetas gráficas (GPU). A diferencia de los métodos geométricos que requieren ordenar los objetos antes de dibujarlos, este algoritmo permite procesar las primitivas en cualquier orden arbitrario. Para lograrlo, el sistema mantiene en la memoria de video dos matrices de datos de las mismas dimensiones que la resolución de la pantalla:

1. Frame Buffer (Búfer de Color): Almacena los valores RGB (color) finales que se mostrarán en cada píxel.
2. Z-Buffer (Búfer de Profundidad): Almacena un valor de punto flotante que representa la distancia (coordenada Z) del objeto más cercano encontrado hasta ese momento para cada píxel. Inicialmente, este búfer se "limpia" con el valor de profundidad máximo posible (usualmente 1.0, representando el plano más lejano).

El funcionamiento es una comprobación condicional constante durante la rasterización. Cuando el motor gráfico intenta dibujar un píxel de un nuevo polígono en la coordenada (x, y), primero calcula su profundidad z. Luego, compara este valor z con el valor almacenado actualmente en el Z-Buffer en la posición (x, y). Si el nuevo valor z es menor (indicando que el nuevo punto está más cerca de la cámara), el sistema actualiza el Z-Buffer con la nueva profundidad y sobrescribe

el color en el Frame Buffer. Si el valor es mayor, el píxel se descarta silenciosamente porque está oculto tras algo ya dibujado.

Una consideración crítica en este método, citada por Foley, es la precisión de profundidad. La resolución del Z-Buffer no es infinita (típicamente 16 o 24 bits), y la distribución de los valores Z no es lineal; hay mucha más precisión cerca de la cámara que lejos de ella. Esto puede causar errores visuales conocidos como *Z-fighting* cuando dos superficies están muy juntas. Para la eliminación de líneas ocultas (el objetivo de esta investigación), este problema se agrava al dibujar líneas de alambre sobre superficies sólidas. Para solucionarlo, se emplea la técnica de *Polygon Offset*, que añade un valor epsilon matemático a la profundidad de las líneas, "engañando" al Z-Buffer para que crea que las líneas están ligeramente más cerca que la superficie sólida, garantizando que se dibujen encima sin ser ocultadas incorrectamente por el propio polígono al que pertenecen [2].

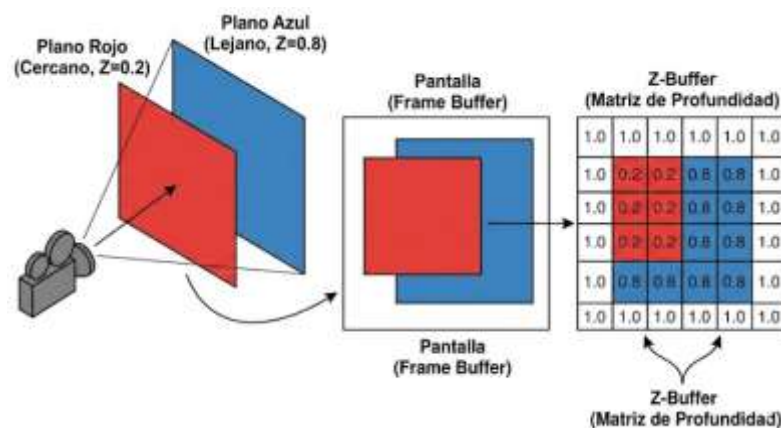


Figura 3. Funcionamiento del Z-Buffer

4.3. Algoritmo de Warnock (Subdivisión de Área)

El algoritmo de Warnock, desarrollado por John Warnock en 1969, representa uno de los primeros y más elegantes enfoques de "divide y vencerás" para resolver el problema de las superficies

ocultas. A diferencia de los métodos que calculan la visibilidad píxel por píxel desde el inicio, este algoritmo trabaja en el espacio imagen considerando áreas rectangulares de la pantalla. La premisa fundamental se basa en la coherencia de área: es muy probable que una región pequeña de la pantalla esté ocupada por un solo polígono o por ninguno. El algoritmo examina una ventana (inicialmente toda la pantalla) y verifica si la escena dentro de esa ventana es lo suficientemente simple para ser resuelta directamente. Si la complejidad de los polígonos dentro del área impide una decisión inmediata sobre qué es visible, la ventana se subdivide recursivamente en cuatro cuadrantes iguales, repitiendo el proceso de evaluación para cada sub-ventana.

El núcleo lógico del algoritmo reside en la clasificación de los polígonos respecto al área de visualización actual. En cada paso recursivo, los polígonos se clasifican en cuatro categorías: (1) Polígonos disjuntos, que están completamente fuera del área y se descartan; (2) Polígonos contenidos, que están completamente dentro del área; (3) Polígonos que intersectan, que cortan los bordes del área; y (4) Polígonos circundantes, que cubren completamente el área de fondo (el observador está "viendo a través" del polígono). Si un área contiene solo un polígono circundante que es el más cercano al observador, o si el área está vacía, el proceso se detiene y se rellena el área con el color correspondiente. Esta estructura de decisión jerárquica se implementa típicamente mediante un árbol cuaternario (Quadtree), lo que optimiza la gestión de memoria al no tener que almacenar un búfer de profundidad completo para toda la pantalla.

El proceso de subdivisión continúa hasta que se cumple una condición de parada. La condición ideal es encontrar un área "simple" (homogénea), pero en escenas con bordes complejos o muchas superposiciones, la subdivisión puede llegar al nivel del píxel individual. En este punto límite, el algoritmo debe tomar una decisión final: calcular la profundidad en el centro del píxel para determinar el color ganador. Una ventaja notable del algoritmo de Warnock es su capacidad

inherente para manejar el *anti-aliasing*. Al llegar a subdivisiones sub-píxel, es posible promediar los colores de los polígonos que comparten ese espacio diminuto, suavizando los bordes dentados típicos de los gráficos rasterizados, una característica que otros algoritmos como el Z-Buffer no ofrecen de forma nativa sin costes computacionales adicionales [2].

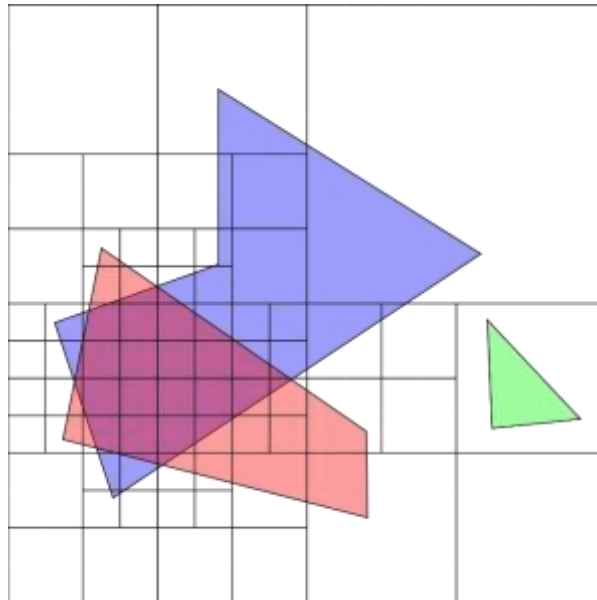


Figura 4. Subdivisión recursiva de área (Algoritmo de Warnock).

4.4. Algoritmo de Línea de Barrido (Scan-Line)

El algoritmo de línea de barrido es una técnica de espacio imagen que procesa la escena fila por fila (línea de escaneo), aprovechando al máximo la coherencia de barrido. En lugar de proyectar cada polígono individualmente sobre la pantalla en un orden arbitrario, este método avanza secuencialmente desde la parte superior de la ventana de visualización hacia la inferior. Para cada línea horizontal (scan-line), el algoritmo calcula las intersecciones con las aristas de todos los polígonos presentes. Estas intersecciones generan una serie de tramos o intervalos horizontales. La tarea del algoritmo se reduce entonces a determinar, para cada intervalo entre dos

intersecciones, cuál polígono es el que se encuentra más cerca del observador y, por lo tanto, cuál debe ser dibujado en ese segmento de la línea.

Para gestionar eficientemente la geometría, este algoritmo utiliza dos estructuras de datos dinámicas fundamentales: la Tabla de Aristas (Edge Table - ET) y la Tabla de Aristas Activas (Active Edge Table - AET). La ET contiene todas las aristas de la escena clasificadas por su coordenada 'Y' máxima, es decir, dónde empiezan a ser relevantes. A medida que la línea de barrido avanza hacia abajo, las aristas que "tocan" la línea actual se mueven de la ET a la AET. La AET mantiene las aristas ordenadas por su coordenada 'X' actual. Gracias a la coherencia espacial, no es necesario recalcular la intersección completa geométricamente en cada paso; simplemente se actualiza la coordenada X de cada arista activa sumándole la inversa de la pendiente ($1/m$) al pasar de una línea de barrido a la siguiente. Esto reduce operaciones costosas de división y multiplicación a simples sumas incrementales, otorgando al algoritmo una gran velocidad en software.

La resolución de la visibilidad ocurre cuando múltiples polígonos se superponen en una misma línea de barrido. El algoritmo mantiene una "bandera" o contador de paridad para saber si está entrando o saliendo de un polígono. Cuando dos o más intervalos de polígonos se solapan en la misma sección de la línea de barrido, se evalúa la profundidad (Z) en ese punto específico para determinar el ganador. A diferencia del Z-Buffer, que requiere memoria para cada píxel de la pantalla, el algoritmo de línea de barrido solo necesita memoria suficiente para procesar la línea actual y mantener las tablas de aristas, lo que lo hizo extremadamente popular en las décadas pasadas cuando la memoria RAM era un recurso escaso. Además, su naturaleza secuencial facilita la integración con técnicas de sombreado suave como Gouraud o Phong, calculando la iluminación simultáneamente con la visibilidad [1].

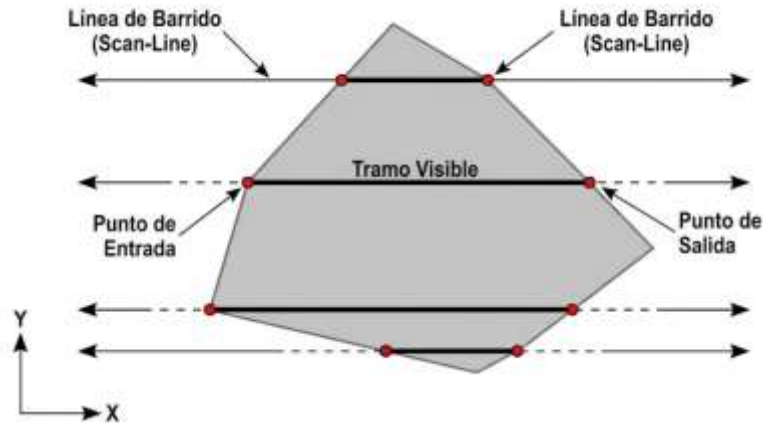


Figura 5. Proceso de barrido de línea (Scan-Line)

V. MODELO MATEMÁTICO Y ALGORÍTMICO

5.1. Modelo Geométrico del Problema

La Eliminación de Líneas Ocultas busca determinar qué segmentos de aristas 3D son visibles desde un punto de vista específico y cuáles deben omitirse porque están ocultos por superficies del objeto.

Se parte de un modelo tridimensional definido por:

- **Vértices:**

$$P_i = (x_i, y_i, z_i)$$

- **Aristas:** segmentos que conectan pares de vértices.
- **Caras (polígonos):** generalmente triángulos o cuadriláteros formados por vértices.

5.2. Sistema de referencia y vector de visión

5.2.1. Vector de visión (cámara)

El observador o cámara se define mediante un punto:

$$C = (x_c, y_c, z_c)$$

Para cualquier punto del objeto P , el vector de visión es:

$$V^{\rightarrow} = P - C$$

Este vector indica la dirección desde la cámara hacia el objeto.

5.3. Vectores normales de las superficies

Cada cara plana del objeto posee un vector normal, el cual es perpendicular a la superficie.

Dada una cara definida por tres vértices:

$$P_1, P_2, P_3$$

Se definen dos vectores del plano:

$$\vec{A} = P_2 - P_1$$

$$\vec{B} = P_3 - P_1$$

El vector normal se obtiene mediante el producto cruz:

$$\vec{N} = \vec{A} \times \vec{B}$$

Expansión del producto cruz:

$$\vec{N} = \begin{pmatrix} i & j & k \\ A_x & A_y & A_z \\ B_x & B_y & B_z \end{pmatrix}$$

Resultado:

$$N^{\rightarrow} = (A_y B_z - A_z B_y, A_z B_x - A_x B_z, A_x B_y - A_y B_x)$$

5.4.Producto punto y Criterio de visibilidad

El producto punto entre dos vectores se define como:

$$\vec{N} \cdot \vec{V} = |\vec{N}| |\vec{V}| \cos(\theta)$$

Donde:

- θ es el ángulo entre la normal y el vector de visión.

5.4.1. Interpretación Geométrica

Resultado del producto punto	Interpretación
$\vec{N} \cdot \vec{V} > 0$	La cara mira hacia la cámara
$\vec{N} \cdot \vec{V} < 0$	La cara está orientada en sentido contrario
$\vec{N} \cdot \vec{V} = 0$	Cara perpendicular a la visión

5.5.Back-Face Culling (Base matemática del HLR)

El Back-Face Culling es el método matemático más simple y eficiente para eliminación de líneas ocultas.

5.5.1. Principio Matemático

Una cara es invisible si:

$$N^{\rightarrow} \cdot V^{\rightarrow} \geq 0$$

Una cara es visible si:

$$N \cdot V < 0$$

5.6.Determinación de aristas visibles

Una arista está formada por dos caras adyacentes:

- Cara F_1 con normal N_1
- Cara F_2 con normal N_2

5.7.Proyección al plano de la imagen

Para representar el objeto en 2D, los puntos 3D se proyectan al plano de la cámara.

5.7.1. Proyección Perspectiva

Si el plano de proyección está a una distancia d :

$$x' = \frac{x}{z} \cdot d$$

$$y' = \frac{y}{z} \cdot d$$

Esto permite:

- Convertir aristas visibles 3D en **segmentos 2D**
- Dibujar únicamente las líneas no ocultas

VI. CONCLUSIONES

La investigación realizada sobre la eliminación de líneas ocultas permite concluir que:

1. La eliminación de líneas ocultas es indispensable para la interpretación correcta de volúmenes en un espacio 3D proyectado en 2D. Sin este proceso, la ambigüedad visual impide la utilidad práctica de los modelos gráficos en ingeniería y diseño.
2. Existe una dicotomía fundamental entre precisión y velocidad. Los algoritmos de espacio objeto ofrecen precisión matemática infinita ideal para planos, mientras que los de espacio imagen (como el Z-Buffer) priorizan la velocidad de renderizado, siendo el estándar actual gracias a la aceleración por hardware de las tarjetas gráficas (GPU).
3. La elección del algoritmo adecuado depende del contexto de la aplicación. Para la implementación práctica en esta asignatura, el uso de OpenGL favorece el uso del Z-Buffer debido a su integración nativa, lo que simplifica la programación al abstraer la complejidad de la ordenación geométrica manual.

VII. RECOMENDACIONES

1. Para la implementación práctica en Visual Code con OpenGL, se recomienda utilizar el método de Z-Buffer (Depth Test) habilitando la funcionalidad `GL_DEPTH_TEST`. Es el método más robusto para escenas dinámicas.
2. Al renderizar líneas sobre polígonos sólidos para visualizar la malla ("wireframe on shaded"), se recomienda investigar y aplicar la técnica de Polygon Offset. Esto evita el "z-fighting", un defecto visual donde las líneas parpadean o desaparecen

incorrectamente debido a que tienen la misma profundidad matemática que la superficie que cubren.

3. Se sugiere comenzar la implementación con formas convexas simples (como un cubo) para verificar el funcionamiento de la detección de caras traseras (Back-Face Culling) antes de intentar escenas complejas con múltiples objetos.

VIII. REFERENCIAS BIBLIOGRÁFICAS

- [1] D. Hearn y M. P. Baker, *Computer graphics, C version*, 2. ed. Upper Saddle River, NJ: Prentice Hall, 2014.
- [2] J. D. Foley, *Computer Graphics: Principles and Practice*. Addison-Wesley Professional, 1996.

IX. ANEXOS

```
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

# --- DEFINICIÓN DE GEOMETRÍA DEL CUBO ---

# Vértices: Las 8 esquinas del cubo (Coordenadas x, y, z)
vertices = (
    (1, -1, -1), (1, 1, -1), (-1, 1, -1), (-1, -1, -1), # Cara
    trasera
    (1, -1, 1), (1, 1, 1), (-1, -1, 1), (-1, 1, 1)      # Cara
    frontal
)

# Aristas: Pares de vértices que forman las líneas (el "alambre")
aristas = (
    (0,1), (0,3), (0,4), (2,1), (2,3), (2,7),
    (6,3), (6,4), (6,7), (5,1), (5,4), (5,7)
)

# Superficies: Grupos de 4 vértices que forman las caras sólidas
# El orden es importante para que OpenGL calcule bien las normales
```

```

superficies = (
    (0,1,2,3), (3,2,7,6), (6,7,5,4),
    (4,5,1,0), (1,5,7,2), (4,0,3,6)
)

def dibujar_cubo():
    """
    Función encargada de la lógica de dibujo.
    Aplica el algoritmo de eliminación de líneas ocultas.
    """

    # --- PARTE 1: MÁSCARA DE PROFUNDIDAD ---
    # Dibujamos el cubo SÓLIDO en color NEGRO (igual al fondo).
    # Esto "llena" el Z-Buffer para tapar lo que esté detrás.

    glEnable(GL_POLYGON_OFFSET_FILL) # Activar desplazamiento
    glPolygonOffset(1.0, 1.0)         # Empujar geometría al fondo

    glBegin(GL_QUADS)
    glColor3fv((0, 0, 0)) # Color negro
    for superficie in superficies:
        for vertice in superficie:
            glVertex3fv(vertices[vertice])
    glEnd()

    glDisable(GL_POLYGON_OFFSET_FILL)

    # --- PARTE 2: LÍNEAS VISIBLES ---
    # Dibujamos las líneas de color VERDE.
    # El Z-Buffer ocultará las que estén detrás de la máscara negra.

    glBegin(GL_LINES)
    glColor3fv((0, 1, 0)) # Color verde
    for arista in aristas:
        for vertice in arista:
            glVertex3fv(vertices[vertice])
    glEnd()

def main():
    """
    Función principal que configura la ventana y controla el bucle.
    Llama a las otras funciones necesarias.
    """

    # 1. Inicializar Pygame y Ventana
    pygame.init()
    dimensiones_pantalla = (800, 600)
    pygame.display.set_mode(dimensiones_pantalla, DOUBLEBUF |
OPENGL)

    # 2. Configurar la Cámara (OpenGL)
    gluPerspective(45,
(dimensiones_pantalla[0]/dimensiones_pantalla[1]), 0.1, 50.0)

```

```

glTranslatef(0.0, 0.0, -5) # Alejar la cámara

# 3. Activar el Algoritmo Z-Buffer
glEnable(GL_DEPTH_TEST)

# 4. Bucle Principal del programa
while True:
    # Control de eventos para cerrar la ventana
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            pygame.quit()
            quit()

    # Rotación automática del cubo (1 grado por vuelta)
    glRotatef(1, 1, 1, 0)

    # Limpiar pantalla y buffer de profundidad
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

    # --- LLAMADA A LA FUNCIÓN DE DIBUJO ---
    dibujar_cubo()

    # Actualizar la pantalla
    pygame.display.flip()
    pygame.time.wait(10)

```

